

Phase 7: Project Documentation

Project Executable Files & Deployment Guide

1. Overview

This document specifies the execution files, runtime commands, environment configurations, and verification steps necessary to run and test ComicCraft locally.

2. Environment Prerequisites

- Python: Version 3.10, 3.11, or 3.12 installed on system.
- pip: Python package manager.
- Git: For version control management.

3. Step-by-Step Installation & Execution

Step 1: Clone the Repository & Enter Workspace

```
git clone <repository-url>
cd magesh
```

Step 2: Set Up Virtual Environment (Recommended)

For macOS / Linux:

```
python3 -m venv env
source env/bin/activate
```

For Windows:

```
python -m venv env
env\Scripts\activate
```

Step 3: Install Required Libraries

```
pip install -r requirements.txt
```

Step 4: Configure Environment Variables

Copy .env.example to .env and insert your Gemini API Key:

```
cp .env.example .env
```

Inside .env:

```
GEMINI_API_KEY=your_gemini_api_key_here
HF_API_KEY=your_huggingface_api_key_here
HOST=127.0.0.1
PORT=8000
```

(Note: If no API key is set, ComicCraft will automatically run in high-fidelity offline fallback mode, generating personalized 5-panel comic storylines and illustrations without failing!)

Step 5: Start the FastAPI Server

```
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

4. Accessing Application Endpoints

Interface	URL	Purpose
ComicCraft Web Application	http://127.0.0.1:8000/	Main user interface with interactive creation form, story presets, and comic preview.
Interactive API Documentation	http://127.0.0.1:8000/docs	Swagger UI allowing direct interactive testing of all REST API endpoints.
ReDoc Documentation	http://127.0.0.1:8000/redoc	Clean technical API reference.
Image Test Utility	http://127.0.0.1:8000/test-image?prompt=Fox+in+forest	Standalone verification of the visual inking engine.
Export Success Page	http://127.0.0.1:8000/export-success	Download confirmation screen.

5. Automated Health & Integration Test Command

To verify that all components (Gemini outline, story, inking, layout, and PDF compilation) are functioning with zero errors, run:

```
python3 -c "from starlette.testclient import TestClient; from app.main import app; c = TestClient(app); assert c.get('/').status_code == 200; print('Server is Healthy!')"
```